

Lab 16- Laplacian Filtering

June 7, 2025

1 Theory

The Laplacian filter is a second-order derivative operator used in image processing to highlight regions of rapid intensity change, making it effective for edge detection. The standard Laplacian kernel is:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

This kernel computes the difference between a pixel and its neighbors, emphasizing areas where the intensity changes sharply. Variants of the Laplacian filter, such as:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

are used to control the sensitivity and directionality of edge detection and sharpening. By subtracting the Laplacian-filtered image from the original, we can enhance edges and produce a sharpened image.

```
[6]: import numpy as np
      from PIL import Image
      import matplotlib.pyplot as plt
```

```
[7]: def Laplacian_filter(input_image):
      img = input_image.resize((400,400), Image.Resampling.LANCZOS)

      fig = plt.figure()
      fig.set_figheight(20)
      fig.set_figwidth(20)

      #plotting original image
      fig.add_subplot(1,2,1)
      plt.imshow(img, cmap='gray')
      plt.title('original')
```

```

# convert to numpy array
numpy_image = np.array(img)
# array for padding
array_b = np.zeros((402,402))

# to pad initial array with zeros in all side
array_b[1:401,1:401] = numpy_image

#defining filter
filter_array = np.array([[0,1,0],
                        [1,-4,1],
                        [0,1,0]])

#creating empty list
lst = []

for i in range(400):
    for j in range(400):
        #extracting part of array equal to filter size
        array_c = array_b[i:(3+i),j:(3+j)]

        #applying filter
        array_mul = np.multiply(filter_array,array_c)
        array_sum = np.sum(array_mul)

        # putting calculated value in list
        lst.append(array_sum)

# resizing lst to shape of original array
final_array = np.resize(lst,(400,400))

final_image = Image.fromarray(final_array)
final_image= final_image.convert("L")

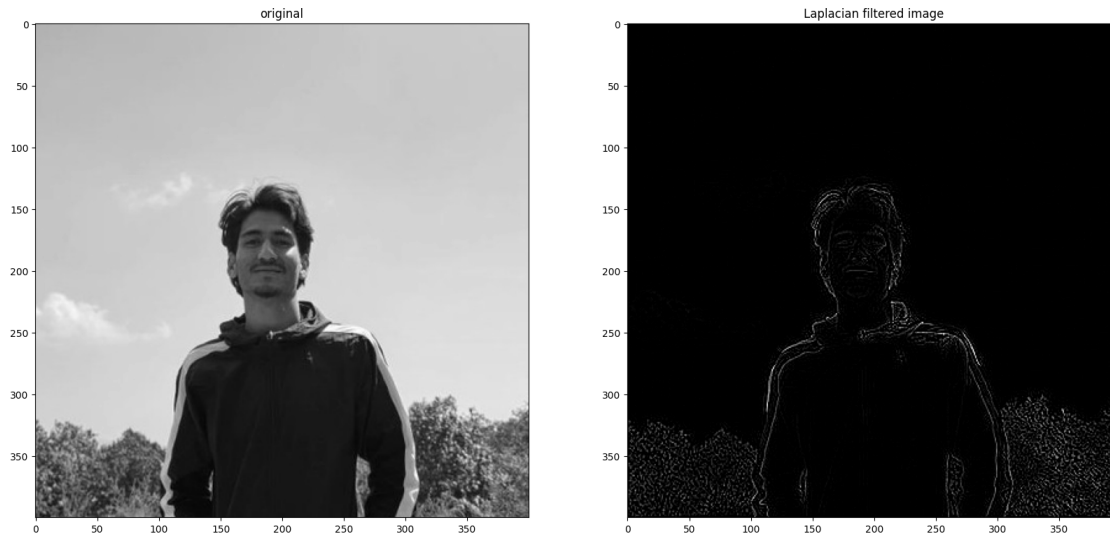
#plotting filtered image
fig.add_subplot(1,2,2)
plt.imshow(final_image, cmap='gray')
plt.title('Laplacian filtered image')

```

```

[8]: # reading image and converting to gray scale
img = Image.open('img/prakash.jpg').convert('L')
# Calling function
Laplacian_filter(img)

```



Variants of Laplace

```
[9]: def Sharpened_Laplacian_filter(input_image):
    img = input_image.resize((400,400), Image.Resampling.LANCZOS)

    fig = plt.figure()
    fig.set_figheight(20)
    fig.set_figwidth(20)

    #plotting original image
    fig.add_subplot(1,2,1)
    plt.imshow(img, cmap='gray')
    plt.title('original')

    # convert to numpy array
    numpy_image = np.array(img)
    # array for padding
    array_b = np.zeros((402,402))

    # to pad initial array with zeros in all side
    array_b[1:401,1:401] = numpy_image

    #defining filter
    filter_array = np.array([[0,-1,0],
                             [-1,5,-1],
                             [0,-1,0]])

    #creating empty list
```

```

lst = []

for i in range(400):
    for j in range(400):
        #extracting part of array equal to filter size
        array_c = array_b[i:(3+i),j:(3+j)]

        #applying filter
        array_mul = np.multiply(filter_array,array_c)
        array_sum = np.sum(array_mul)

        # putting calculated value in list
        lst.append(array_sum)

# resizing lst to shape of original array
final_array = np.resize(lst,(400,400))

final_image = Image.fromarray(final_array)
final_image= final_image.convert("L")

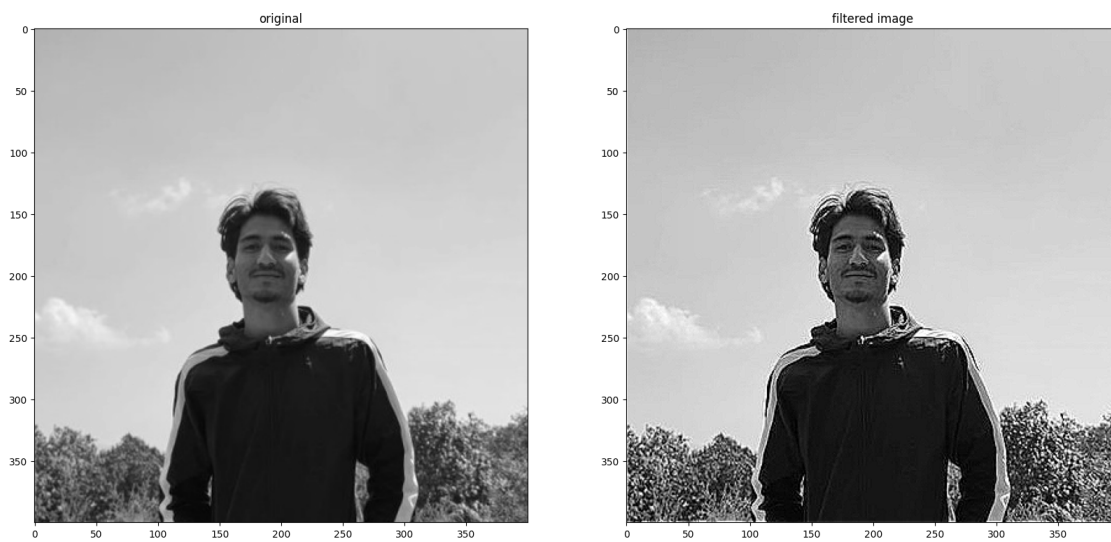
#plotting filtered image
fig.add_subplot(1,2,2)
plt.imshow(final_image, cmap='gray')
plt.title('filtered image')

```

```

[10]: # reading image and converting to gray scale
img = Image.open('img/prakash.jpg').convert('L')
# Calling function
Sharpened_Laplacian_filter(img)

```



```

[11]: def Laplacian_Variant1(input_image):
    img = input_image.resize((400,400), Image.Resampling.LANCZOS)

    fig = plt.figure()
    fig.set_figheight(20)
    fig.set_figwidth(20)

    #plotting original image
    fig.add_subplot(1,2,1)
    plt.imshow(img, cmap='gray')
    plt.title('original')

    # convert to numpy array
    numpy_image = np.array(img)
    # array for padding
    array_b = np.zeros((402,402))

    # to pad initial array with zeros in all side
    array_b[1:401,1:401] = numpy_image

    #defining filter
    filter_array = np.array([[1,1,1],
                             [1,-8,1],
                             [1,1,1]])

    #creating empty list
    lst = []

    for i in range(400):
        for j in range(400):
            #extracting part of array equal to filter size
            array_c = array_b[i:(3+i),j:(3+j)]

            #applying filter
            array_mul = np.multiply(filter_array,array_c)
            array_sum = np.sum(array_mul)

            # putting calculated value in list
            lst.append(array_sum)

    # resizing lst to shape of original array
    final_array = np.resize(lst,(400,400))

    final_image = Image.fromarray(final_array)

```

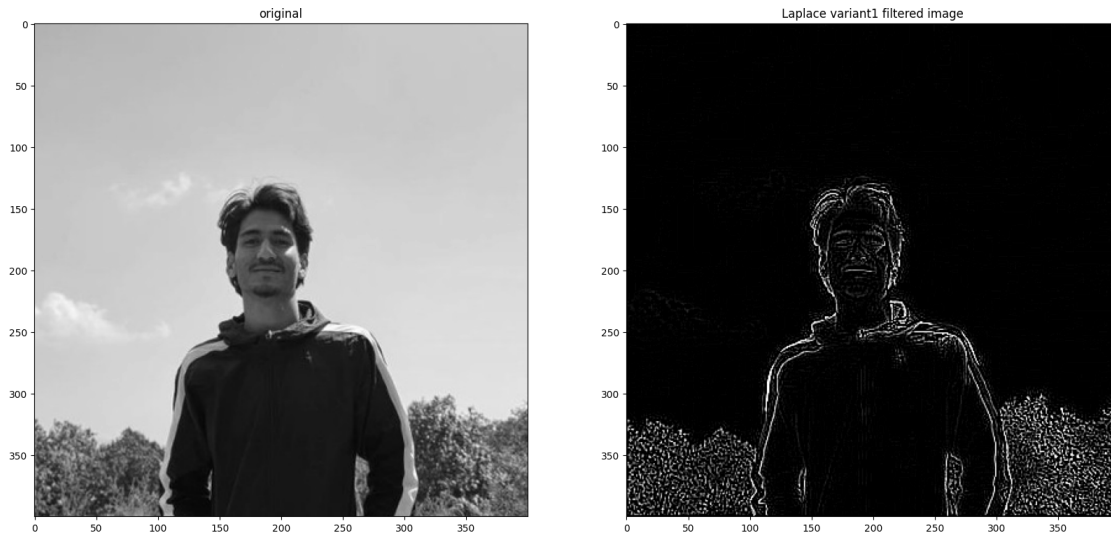
```

final_image= final_image.convert("L")

#plotting filtered image
fig.add_subplot(1,2,2)
plt.imshow(final_image, cmap='gray')
plt.title('Laplace variant1 filtered image')

```

[12]: Laplacian_Variant1(img)



```

[13]: def Laplacian_Variant2(input_image):
    img = input_image.resize((400,400), Image.Resampling.LANCZOS)

    fig = plt.figure()
    fig.set_figheight(20)
    fig.set_figwidth(20)

    #plotting original image
    fig.add_subplot(1,2,1)
    plt.imshow(img, cmap='gray')
    plt.title('original')

    # convert to numpy array
    numpy_image = np.array(img)
    # array for padding
    array_b = np.zeros((402,402))

    # to pad initial array with zeros in all side

```

```

array_b[1:401,1:401] = numpy_image

#defining filter
filter_array = np.array([[ -1, -1, -1],
                        [ -1, 9, -1],
                        [ -1, -1, -1]])

#creating empty list
lst = []

for i in range(400):
    for j in range(400):
        #extracting part of array equal to filter size
        array_c = array_b[i:(3+i),j:(3+j)]

        #applying filter
        array_mul = np.multiply(filter_array,array_c)
        array_sum = np.sum(array_mul)

        # putting calculated value in list
        lst.append(array_sum)

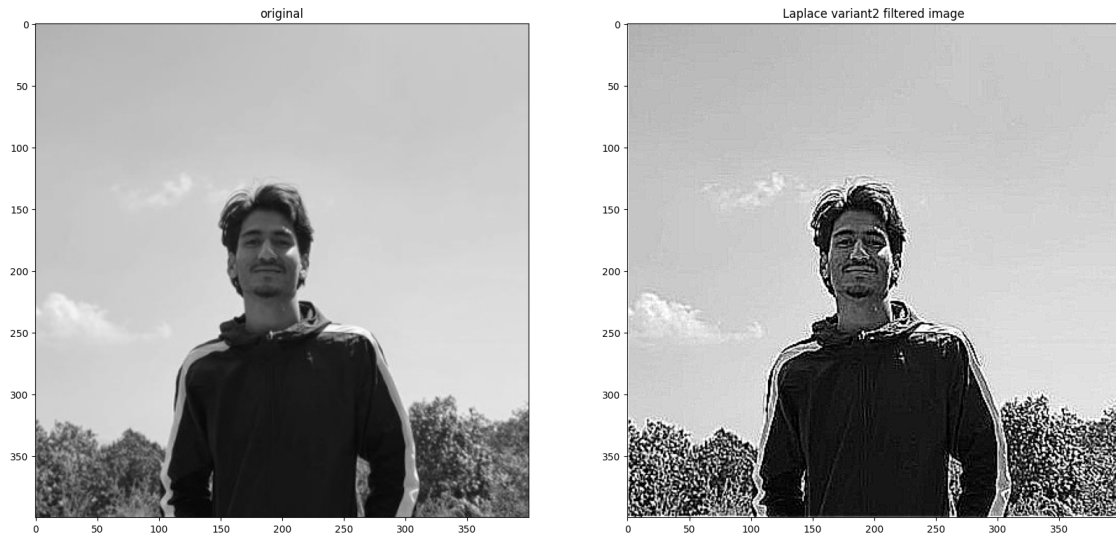
# resizing lst to shape of original array
final_array = np.resize(lst,(400,400))

final_image = Image.fromarray(final_array)
final_image= final_image.convert("L")

#plotting filtered image
fig.add_subplot(1,2,2)
plt.imshow(final_image, cmap='gray')
plt.title('Laplace variant2 filtered image')

```

```
[14]: Laplacian_Variant2(img)
```



2 Conclusion

Laplacian filters and their variants are powerful tools for edge detection and image sharpening. The choice of kernel affects the strength and type of enhancement. Applying these filters highlights image details and improves visual quality, making them essential in image analysis and computer vision tasks.